

Programación Web

# Taller de Programación Web: API de Comercio Electrónico para Tienda Universitaria

---

Enunciado formal del taller con alcance funcional, reglas de negocio, historias de usuario, clases obligatorias, pruebas y rúbrica.

## Stack obligatorio

Java 21 · Spring Boot 4 · PostgreSQL · Testcontainers · JUnit 5 · Mockito

Documento de enunciado formal, criterios de evaluación y especificación técnica.

## 1. Contexto del problema

La universidad desea digitalizar su tienda institucional, donde vende productos como sudaderas, libros, kits académicos, cuadernos, accesorios y artículos promocionales. El proceso actual presenta problemas de sobreventa, errores en inventario, falta de trazabilidad en el ciclo del pedido y poca capacidad de análisis comercial.

Se requiere una API REST que gestione clientes, direcciones, categorías, productos, inventario, pedidos y detalle de pedidos, incorporando reglas de negocio reales, consistencia transaccional y consultas orientadas a la operación y al análisis.

## 2. Objetivo general

Diseñar e implementar una aplicación backend con arquitectura por capas, base de datos relacional y pruebas automatizadas, utilizando el stack académico definido para el curso.

## 3. Objetivos específicos

- Diseñar un backend para ventas e inventario con Java 21, Spring Boot 4, PostgreSQL y pruebas automatizadas.
- Implementar repositorios con consultas derivadas, JPQL y filtros compuestos, probados con Testcontainers sobre PostgreSQL real.
- Implementar lógica de negocio en la capa service para controlar stock, cálculo de montos y transiciones de estado del pedido.
- Implementar una API REST validada y probada con MockMvc y manejo global de excepciones.

## 4. Alcance funcional

- Registrar clientes, direcciones, categorías y productos.
- Administrar el inventario y el stock mínimo de cada producto.
- Crear pedidos con múltiples ítems y calcular subtotales y total automáticamente.
- Gestionar cambios de estado del pedido: CREATED, PAID, SHIPPED, DELIVERED y CANCELLED.
- Consultar productos y pedidos por filtros y construir reportes de ventas e inventario.

## Consultas mínimas esperadas en la capa repository

### Consultas derivadas (Query Methods)

- Buscar producto por SKU.
- Buscar productos activos por categoría.
- Buscar pedidos por cliente.
- Buscar productos con bajo stock.

### Consultas JPQL / agregación

- Buscar pedidos por filtros combinados: cliente, estado, rango de fechas, total mínimo y total máximo.
- Productos más vendidos por período.

- Ingresos mensuales agrupados.
- Clientes con mayor facturación.
- Productos con stock insuficiente respecto al mínimo.
- Historial de cambios de un pedido.
- Top de categorías por volumen de ventas.

## **Lógica de negocio obligatoria en la capa service**

### **6.1 Reglas para productos e inventario**

- Todo producto debe tener un SKU único dentro del sistema.
- Todo producto debe pertenecer a una categoría existente.
- El precio del producto debe ser mayor que cero.
- El inventario disponible y el stock mínimo no pueden ser negativos.
- No se puede desactivar un producto que forme parte de pedidos activos, salvo que la regla de negocio adicional lo permita explícitamente.

### **6.2 Reglas para creación de pedidos**

- Un pedido debe asociarse a un cliente existente y activo.
- La dirección de envío debe pertenecer al cliente del pedido.
- El pedido debe contener al menos un ítem.
- No se permiten cantidades menores o iguales a cero.
- Cada producto del pedido debe existir y estar activo.
- El precio unitario del ítem se toma del producto al momento de la creación del pedido.
- El subtotal del ítem se calcula como quantity por unitPrice.
- El total del pedido se calcula como la suma de subtotales.
- El cliente no puede enviar el total del pedido de forma manual.
- Todo pedido nuevo debe crearse con estado inicial CREATED.

### **6.3 Reglas para pago del pedido**

- Solo un pedido en estado CREATED puede pasar a PAID.
- Antes de pagar, el sistema debe validar stock suficiente para todos los ítems.
- Si un solo ítem no tiene stock suficiente, todo el pago debe rechazarse.
- Al pasar a PAID, el servicio debe descontar el inventario disponible de cada producto.
- Todo cambio de estado debe registrarse en el historial del pedido.

### **6.4 Reglas para envío y entrega**

- Solo un pedido PAID puede pasar a SHIPPED.
- No se puede enviar un pedido cancelado.
- Solo un pedido SHIPPED puede pasar a DELIVERED.
- No se puede marcar como entregado un pedido que nunca fue despachado.

## 6.5 Reglas para cancelación

- Un pedido CREATED puede cancelarse sin reversión de stock, porque aún no se ha descontado inventario.
- Un pedido PAID puede cancelarse y debe revertir el stock descontado.
- Un pedido SHIPPED no podrá cancelarse si esa es la regla definida para el taller.
- Un pedido DELIVERED no puede cancelarse.
- Toda cancelación debe dejar trazabilidad en el historial del pedido.

## 6.6 Reglas de consulta y reportes

- Debe poder consultarse productos con bajo stock.
- Debe poder consultarse pedidos por cliente, estado, rango de fechas, total mínimo y total máximo.
- Deben generarse reportes de productos más vendidos, ingresos mensuales, clientes con mayor facturación y productos bajo stock.

## Historias de usuario

| Código | Historia de usuario                                                                                                  |
|--------|----------------------------------------------------------------------------------------------------------------------|
| HU-01  | Como administrador, quiero registrar categorías para clasificar los productos de la tienda.                          |
| HU-02  | Como administrador, quiero registrar productos con SKU, precio y categoría para ponerlos a disposición del catálogo. |
| HU-03  | Como administrador, quiero administrar el inventario de los productos para controlar existencias y stock mínimo.     |
| HU-04  | Como cliente, quiero registrar mis datos para poder realizar pedidos.                                                |
| HU-05  | Como cliente, quiero registrar direcciones de envío para recibir mis compras.                                        |
| HU-06  | Como usuario del sistema, quiero crear un pedido con varios productos para formalizar una compra.                    |
| HU-07  | Como usuario del sistema, quiero que el total del pedido se calcule automáticamente para evitar errores manuales.    |
| HU-08  | Como administrador, quiero confirmar el pago de un pedido validando inventario disponible para evitar sobreventa.    |
| HU-09  | Como administrador, quiero despachar un pedido pagado para avanzar en el flujo de entrega.                           |
| HU-10  | Como administrador, quiero marcar un pedido como entregado para cerrar el ciclo comercial.                           |
| HU-11  | Como administrador, quiero cancelar pedidos cuando corresponda y revertir inventario si ya había sido descontado.    |

| Código | Historia de usuario                                                                                                            |
|--------|--------------------------------------------------------------------------------------------------------------------------------|
| HU-12  | Como coordinador de tienda, quiero consultar reportes de ventas, top productos y bajo stock para tomar decisiones comerciales. |

## Diseño mínimo de base de datos

| Tabla                | Propósito funcional                                                           |
|----------------------|-------------------------------------------------------------------------------|
| customers            | Almacena los datos del cliente y su estado de actividad.                      |
| addresses            | Guarda una o varias direcciones de envío asociadas a un cliente.              |
| categories           | Catálogo de categorías para clasificar los productos.                         |
| products             | Representa el catálogo de productos disponibles para venta.                   |
| inventories          | Controla stock disponible, stock mínimo y relación uno a uno con el producto. |
| orders               | Representa el pedido, su cliente, dirección, estado, fechas y total.          |
| order_items          | Detalla los productos, cantidades, precios y subtotales de cada pedido.       |
| order_status_history | Registra la trazabilidad de cambios de estado del pedido.                     |

## Relaciones mínimas

- Un cliente tiene muchas direcciones.
- Un cliente tiene muchos pedidos.
- Una categoría tiene muchos productos.
- Un producto tiene un inventario.
- Un pedido tiene muchos ítems.
- Un pedido tiene muchos cambios de estado.

## Endpoints mínimos a implementar

| Módulo      | Endpoints                                                                                                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Clientes    | <ul style="list-style-type: none"> <li>• POST /api/customers</li> <li>• GET /api/customers/{id}</li> <li>• GET /api/customers</li> <li>• PUT /api/customers/{id}</li> </ul> |
| Direcciones | <ul style="list-style-type: none"> <li>• POST /api/customers/{customerId}/addresses</li> <li>• GET /api/customers/{customerId}/addresses</li> </ul>                         |
| Categorías  | <ul style="list-style-type: none"> <li>• POST /api/categories</li> <li>• GET /api/categories</li> </ul>                                                                     |

| Módulo    | Endpoints                                                                                                                                                                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Productos | <ul style="list-style-type: none"> <li>• POST /api/products</li> <li>• GET /api/products/{id}</li> <li>• GET /api/products</li> <li>• PUT /api/products/{id}</li> <li>• PUT /api/products/{id}/inventory</li> </ul>                                                                     |
| Pedidos   | <ul style="list-style-type: none"> <li>• POST /api/orders</li> <li>• GET /api/orders/{id}</li> <li>• GET /api/orders</li> <li>• PUT /api/orders/{id}/pay</li> <li>• PUT /api/orders/{id}/ship</li> <li>• PUT /api/orders/{id}/deliver</li> <li>• PUT /api/orders/{id}/cancel</li> </ul> |
| Reportes  | <ul style="list-style-type: none"> <li>• GET /api/reports/best-selling-products</li> <li>• GET /api/reports/monthly-income</li> <li>• GET /api/reports/top-customers</li> <li>• GET /api/reports/low-stock-products</li> </ul>                                                          |

### Lista exacta de clases a implementar

| Grupo        | Clases obligatorias                                                                                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Entidades    | Customer<br>Address<br>Category<br>Product<br>Inventory<br>Order<br>OrderItem<br>OrderStatusHistory                                                                                 |
| Enums        | OrderStatus<br>CustomerStatus                                                                                                                                                       |
| Repositories | CustomerRepository<br>AddressRepository<br>CategoryRepository<br>ProductRepository<br>InventoryRepository<br>OrderRepository<br>OrderItemRepository<br>OrderStatusHistoryRepository |
| DTOs request | CreateCustomerRequest<br>UpdateCustomerRequest<br>CreateAddressRequest<br>CreateCategoryRequest<br>CreateProductRequest<br>UpdateProductRequest                                     |

| Grupo                       | Clases obligatorias                                                                                                                                                                                                                            |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             | UpdateInventoryRequest<br>CreateOrderRequest<br>CreateOrderItemRequest<br>CancelOrderRequest                                                                                                                                                   |
| DTOs response               | CustomerResponse<br>AddressResponse<br>CategoryResponse<br>ProductResponse<br>InventoryResponse<br>OrderItemResponse<br>OrderResponse<br>BestSellingProductResponse<br>MonthlyIncomeResponse<br>TopCustomerResponse<br>LowStockProductResponse |
| Services                    | CustomerService<br>AddressService<br>CategoryService<br>ProductService<br>InventoryService<br>OrderService<br>ReportService                                                                                                                    |
| Implementaciones service    | CustomerServiceImpl<br>AddressServiceImpl<br>CategoryServiceImpl<br>ProductServiceImpl<br>InventoryServiceImpl<br>OrderServiceImpl<br>ReportServiceImpl                                                                                        |
| Controllers                 | CustomerController<br>AddressController<br>CategoryController<br>ProductController<br>OrderController<br>ReportController                                                                                                                      |
| Mappers                     | CustomerMapper<br>AddressMapper<br>CategoryMapper<br>ProductMapper<br>OrderMapper                                                                                                                                                              |
| Excepciones y manejo global | ResourceNotFoundException<br>BusinessException<br>ConflictException<br>ValidationException<br>GlobalExceptionHandler                                                                                                                           |

## Pruebas obligatorias

| Componente                   | Alcance mínimo esperado                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Repository integration tests | <ul style="list-style-type: none"> <li>• Implementar al menos: ProductRepositoryIntegrationTest, InventoryRepositoryIntegrationTest, OrderRepositoryIntegrationTest y CustomerRepositoryIntegrationTest.</li> <li>• Cubrir como mínimo: búsqueda de productos por categoría, productos con bajo stock, pedidos por filtros compuestos, top productos vendidos, ingresos mensuales y clientes con mayor facturación.</li> </ul> |
| Service unit tests           | <ul style="list-style-type: none"> <li>• Implementar al menos: OrderServiceImplTest, InventoryServiceImplTest y ProductServiceImplTest.</li> <li>• Cubrir como mínimo: no crear pedido sin ítems, no crear pedido con cantidad inválida, cálculo correcto de subtotal y total, rechazo de pago por stock insuficiente, descuento de inventario, reversión de stock y validación de transiciones de estado.</li> </ul>          |
| Controller tests             | <ul style="list-style-type: none"> <li>• Implementar al menos: OrderControllerTest, ProductControllerTest, CustomerControllerTest y ReportControllerTest.</li> <li>• Cubrir como mínimo: creación de pedido válida, errores de validación, error por stock insuficiente, cambios de estado exitosos y reportes funcionales.</li> </ul>                                                                                         |

## Entregables

- Proyecto Maven completo y organizado por capas.
- Diagrama entidad-relación.
- Colección Postman o equivalente para consumo de la API.
- README técnico con reglas de negocio, decisiones de diseño y forma de ejecución.
- Pruebas automáticas funcionando correctamente.

## Rúbrica de evaluación

| Criterio                      | Peso | Descripción                                                                                         |
|-------------------------------|------|-----------------------------------------------------------------------------------------------------|
| Modelo relacional             | 15%  | Calidad de entidades, relaciones, restricciones y consistencia del diseño.                          |
| Repositories y consultas      | 15%  | Uso de Spring Data JPA, consultas derivadas, JPQL y pruebas de integración.                         |
| Lógica de negocio en services | 25%  | Cálculo correcto de totales, control de stock, transiciones de estado y consistencia transaccional. |
| API REST                      | 10%  | Diseño de endpoints, códigos de respuesta, DTOs y validación.                                       |
| Testing                       | 20%  | Cobertura y calidad de unit tests, integration tests y controller tests.                            |



| Criterio           | Peso | Descripción                                                            |
|--------------------|------|------------------------------------------------------------------------|
| Calidad del código | 10%  | Legibilidad, estructura, separación por capas y manejo de excepciones. |
| Documentación      | 5%   | Claridad del README, diagrama y explicación de reglas.                 |

## Observaciones para la entrega

- Se espera una solución funcional, coherente con las reglas de negocio y correctamente separada por capas.
- Las pruebas deben ser ejecutables y estables; no se aceptarán pruebas comentadas o incompletas.
- El diseño del proyecto debe privilegiar claridad, mantenibilidad y consistencia.